

DevSecOps 5-3-Cloud-based-IaC

Task 1 - Introduction

Cloud-based IaC leverages tools like Terraform and AWS CloudFormation to programmatically provision, scale, and secure elastic cloud resources through version-controlled, API-driven automation.

Task 2 - Terraform 101

Terraform is an infrastructure as code tool used for provisioning that allows the user to define both cloud and on-prem resources in a human-readable configuration file that can be versioned, reused and distributed across teams. Terraform uses a consistent workflow to make the management of your infrastructure an easy and reliable process.

Terraform's Architecture:

Terraform Core: As the name suggests, Terraform Core is responsible for the core functionalities that allow users to provision and manage their infrastructure using Terraform. Terraform is a declarative tool (reminder: this means that a desired state is defined, and the tool is responsible for ensuring the infrastructure meets that desired state, rather than the user declaring each step required to achieve the desired state), to ensure the user infrastructure is in the desired state, Terraform Core takes its input from two sources:

- **Terraform Config files:** The user defines the desired state of their infrastructure in these config files, in other words, what resources make up their desired infrastructure.
- **State:** Terraform has a state file that keeps track of the current state of provisioned infrastructure. The core component checks this state file against the desired state defined in the config files, and, if there are resources that are defined but not provisioned (or the other way around), makes a plan of how to take the infrastructure from its current state to the desired state. By default,

this state file is called terraform.tfstate and is stored in the same directory where Terraform runs.

The Terraform Core takes input from two places: the user-defined configuration files and tfstate. Should there be any difference between the desired state (configuration files) and the current state (tfstate), the required actions will be executed using a provider.

Providers: Providers are used to interact with cloud providers, SaaS providers and other APIs. For example, if an AWS resource like an EC2 instance is defined, the AWS provider would be used; if a Kubernetes cluster is defined, the Kubernetes provider would be required.

Terraform is an open-source, provider-based IaC tool that enables DevSecOps teams to programmatically declare, version, and manage both on-premises and cloud resources using a standardized, predictable workflow. Terraform stands out by offering a human-readable, declarative configuration language (HCL) that prevents vendor lock-in through multi-cloud management while enabling seamless version control, team collaboration, and change tracking across infrastructure iterations.

Answer the questions below

Terraform Core takes its input from two sources; this source keeps track of what the infrastructure currently looks like. What is the name of this source?

State

The other source defines what you want infrastructure to look like. What is the name of this source?

Terraform Config files

If there is a difference between what the infrastructure currently looks like and what you want it to look like, this will require a change. This change will be actioned by a....?

Provider

Task 3 - Terraform Configuration

Configuration & Terraform

Terraform config files are written in a declarative language called HCL (HashiCorp Configuration Language), which, is easy to understand and human-readable. The primary purpose of this language is to declare resources; these resources represent infrastructure objects. The combined definition of these resources makes up the desired infrastructure state.

To give you an example of how, as a DevSecOps engineer, Terraform configuration files might be used, let us consider an example where we want to define a simple VPC (virtual private cloud) using the AWS provider. That would look like this:

```
provider "aws" {  
  region = "eu-west-2"  
}  
  
# Create a VPC  
resource "aws_vpc" "flynet_vpc" {  
  cidr_block = "10.0.0.0/16"  
  tags = {  
    Name = "flynet-vpc"  
  }  
}
```

This Terraform snippet configures the AWS provider for the `eu-west-2` region and provisions a Virtual Private Cloud named `flynet_vpc` with a `10.0.0.0/16` CIDR block and a identifying `flynet-vpc` tag.

Resource Relationships: Sometimes, resources can depend on other resources. When defining a resource with dependencies, the resources it depends on will be referenced. For example, if you were assigned a task to define an AWS security group ingress rule, allowing SSH access from any source within the VPC, you would add the following code block to your configuration file.

```
resource "aws_security_group" "example_security_group" {  
  name = "example-security-group"  
  description = "Example Security Group"
```

```

vpc_id = aws_vpc.flynet_vpc.id #Reference to the VPC created
above (format: resource_type.resource_name.id)

# Ingress rule allowing SSH access from any source within th
e VPC
ingress {
  #Since we are allowing SSH traffic , from port and to port
should be set to port 22
  from_port = 22
  to_port = 22
  protocol = "tcp"
  cidr_blocks = [aws_vpc.flynet_vpc.cidr_block]
}
}

```

This HCL snippet provisions an AWS security group attached to `flynet_vpc` via resource referencing, enforcing an ingress rule that restricts SSH access (port 22/TCP) exclusively to traffic originating within the VPC's IP range. (A **VPS (Virtual Private Server)** is a virtual machine sold as a service by an Internet hosting provider. It simulates a dedicated physical server within a shared hardware environment using virtualization technology (such as KVM, VMware, or Xen).)

Infrastructure Modularisation: This is common practice in larger organisations as it simplifies the management of complex infrastructure configurations. Let's take a look at what the above example configuration would look like. Below is a representation of a directory `/tfconfig` and the files within it, with a brief comment explaining the purpose of each file:

```

tfconfig/
  -flynet_vpc_security.tf #Like mentioned, instead of having a
  ll resources defined in one file, resources can be paired up
  and defined in separate modular files
  -other_module.tf
  -variables.tf #some values will be used across two or many i
  nfrastructure modules, so instead of declaring these repeated

```

ly in each .tf file it makes sense to parameterise them in a file called variables.tf. These variables can then be directly referenced in the .tf file.

-main.tf #main.tf acts as the central configuration file where the defined modules are all referenced in one place

If we consider the example given above, if we were to define a vpc using this structure, it could be configured like so: (variables.tf)

```
variable "vpc_cidr_block" {
  description = "CIDR block for the VPC"
  type = string #Set the type of variable (string,number,bool etc)
  default = "10.0.0.0/16" # Can be changed as needed
}
```

We are defining a VPC the same way as we did at the beginning of this task, except now we are storing it in a variables.tf file. We are doing this so we can reuse/reference it. This could be referenced in any module.tf file, for example, below, we have a code block for a module named flynet_vpc_security: (flynet_vpc_security.tf)

```
cidr_block = var.vpc_cidr_block
#Variable can be referenced using var. followed by variable name
```

Finally, this module (and all other module tf files) would be collected and referenced in the main.tf file like so: main.tf

```
module "flynet_vpc_security" {
  source = "../flynet_vpc_security.tf"
  #where module is defined (relative to current file)
}

module "other_module" {
```

```
source = "../0ther_module.tf"  
}
```

This level of organisation would be extreme, given how primitive the infrastructure setup is in our example. However, this was to show you how an infrastructure at scale might be configured in a larger organisation/company.

Answer the questions below

When defining the VPC (in the first example) before the resource block, what was defined?

provider

When modulating your infrastructure, what file acts as the central configuration file?

main.tf

Instead of repeatedly defining values across multiple infrastructure modules these values can be collected in one file and referenced. What is the name of this file?

variables.tf

Task 4 - Terraform Workflow

The Terraform workflow generally follows four steps: Write, Initialise, Plan and Apply. To better understand this workflow and how it can benefit DevSecOps engineers while provisioning of infrastructure, let's consider an infrastructure at different points in development. When dealing with infrastructure provisioning, you tend to have different problems at different stages. To illustrate this, let's consider the Terraform workflow at three points in time:

- Day 1 (the infrastructure does not exist yet and needs to be provisioned from scratch)
- Day 2+ (The infrastructure exists, but some changes need to be made)
- Day N (an undefined point in time when you no longer need this infrastructure)

Day 1: Let's take a look at what the Terraform workflow would look like on Day 1 of infrastructure provisioning. You have an empty environment with no infrastructure set up and need to define and provision this infrastructure. Using Terraform, you would follow this workflow:

- **Write:** In this part of the workflow you would define the desired state of your infrastructure in a Terraform configuration file. As discussed in the previous task, these configuration files are used to define the desired components, and these components can reference each other if they have a relation.
- **Initialise:** The next part of the workflow is to initialise. The `init` command prepares your workspace (the working directory where your Terraform configuration files are) so Terraform can apply your changes. This preparation includes things like downloading dependencies, for example, if your configuration file defines 'aws' as the provider, during the initialisation, Terraform will download the aws provider plug-in. This is done using the following command: `terraform init`
- **Plan:** After initialising your workspace, you should now plan your changes. The planning stage considers what infrastructure is present currently (with it being Day 1, there will be no infrastructure present) vs the desired state and prepares a plan of actions which, if applied, will make it so the infrastructure matches the desired state. It produces this execution plan and will tell the user what components will be added/destroyed in the process. This is done using the following command: `terraform plan`
- **Apply:** Apply actions the steps (detailed in the plan) required to take the infrastructure from the current state to the desired state. Terraform works out which order these infrastructure resources should be created. In our example from the previous task, we defined two resources: a VPC and a security group. Since the security group references the VPC and is therefore dependent on it, Terraform would create the VPC first. This is done using the following command: `< terraform apply`

Day 2+: Day 2+ means any day after the infrastructure has been provisioned; what will the workflow look like when making changes to an existing infrastructure? Say, for instance, you wanted to add a component to your infrastructure.

- **Initialise:** `terraform init` should be the first command run after making any changes to an infrastructure configuration. It will initialise the workspace just like when creating the infra from scratch.
- **Plan:** Terraform plan is not required but is best practice, especially in an already existing infrastructure. The execution plan generated by the plan command will show you exactly what is being removed or added to the infrastructure. This can catch any misconfigurations before they are applied and break anything. The plan will assess the existing infrastructure in this case and note that the existing components match the desired state, so no changes need to be made, but the additional component has not been created yet and will need to be created to match the desired state, so will be listed as a resource that needs to be added.
- **Apply:** Again, Terraform will provision the infrastructure changes defined using the execution plan made during the Plan phase (note that if you skip the planning phase, Terraform will automatically create an execution plan as if you had and will ask you to approve the plan before execution). The state file will then be updated to reflect that the current state now matches the desired state as the additional component has been added/provisioned.

Day N: Day N denotes a day in the future when the infrastructure is no longer needed. Maybe it was just a test environment, or the application it supports has been shelved. Now that this infrastructure is no longer needed, all that is required is to destroy it.

- **Destroy:** Running the `terraform destroy` command will take care of this. Functionally, the destroy command is quite similar to the apply command in that it takes a plan (a destroy plan which Terraform makes on running this command) and compares it to what infrastructure currently exists and actions that plan instead of provisioning/de-provisioning resources changes, it tears down all resources.

Answer the questions below

What command would you run to action the steps outlined to take your infrastructure from the desired state to the actual state?

terraform apply

What command would you run to prepare your workspace?

```
terraform init
```

What command, when run, would provide you with a series of actions that would be required to take your infrastructure from the desired state to the actual state?

```
terraform plan
```

Task 5 - Cloudformation 101

CloudFormation is an Amazon Web Services (AWS) IaC tool for automated provision and resource management. Instead of manually configuring resources through the AWS Management Console or using the AWS Command Line Interface (CLI), you can use CloudFormation templates to describe your infrastructure in a "declarative" manner.

Declarative Infrastructure as Code: With CloudFormation, you express the desired state of your infrastructure using a JSON or YAML template. This template defines the resources, their configurations, and the relationships between them. CloudFormation provides and manages these resources, making managing and replicating your infrastructure easier.

Templates and Stacks: A CloudFormation template is a text file that serves as a blueprint for your infrastructure. It contains sections that describe various AWS resources like EC2 instances, S3 buckets, and more. When you use a template to create resources, it forms a CloudFormation stack. Stacks are the fundamental units of CloudFormation, and they represent a collection of AWS resources that are created, updated, and deleted together.

```
# CloudFormation Template
AWSTemplateFormatVersion: '2010-09-09' # Version of the CloudFormation template syntax

Description: 'A simple CloudFormation template' # Description of the template

Resources: # Section where AWS resources are defined
```

```

MyEC2Instance: # Logical name of the resource
  Type: 'AWS::EC2::Instance' # Type of AWS resource being
created

  Properties: # Properties specific to the resource type
    ImageId: 'ami-12345678' # ID of the Amazon Machine Image (AMI) for the EC2 instance
    InstanceType: 't2.micro' # Type of EC2 instance
    KeyName: 'my-key-pair' # SSH key pair for accessing the instance

MyS3Bucket: # Another resource example
  Type: 'AWS::S3::Bucket'
  Properties:
    BucketName: 'my-s3-bucket' # Name of the S3 bucket

Outputs: # Section to define outputs for the template
  EC2InstanceId: # Logical name for the output
    Description: 'ID of the EC2 instance' # Description of the output
    Value: !Ref MyEC2Instance # Reference to the EC2 instance resource

```

- **AWSTemplateFormatVersion:** This specifies the CloudFormation template version.
- **Description:** Provides a brief description of the template.
- **Resources:** This section defines the AWS resources to be created, such as EC2 instances or S3 buckets. Each resource has a logical name (MyEC2Instance, MyS3Bucket). Type indicates the AWS resource type. Properties hold configuration settings for the resource.
- **Outputs:** This section defines the output values displayed after creating the stack. Logical name, description, and a reference to a resource using `!Ref`.

CloudFormation Architecture: Similar to Terraform, CloudFormation follows a workflow for provision and management. To better understand the workflow, we

need to understand the inner workings of CloudFormation and its Architecture.

Main and Worker Architecture: CloudFormation employs a main-worker architecture. The main, typically a CloudFormation service running in AWS, interprets and processes the CloudFormation template. It manages the overall stack creation, update, or deletion orchestration. The worker nodes, distributed across AWS regions, are responsible for carrying out the actual provisioning of resources.

Template Processing Flow

- **Template Submission:** users submit a CloudFormation template, written in JSON or YAML, to the CloudFormation service. The template specifies the desired AWS resources and their configurations. This can be stored in an S3 bucket, for example.
- **Template Validation:** the CloudFormation service validates the submitted template to ensure its syntax is correct and it follows AWS resource specifications.
- **Processing by the Main Node:** the main node processes the template, creating a set of instructions for resource provisioning. It determines the order in which resources should be created based on dependencies.
- **Resource Provisioning:** the main node communicates with worker nodes distributed across different AWS regions. Worker nodes carry out the actual provisioning of resources stated by the instructions provided by the main.
- **Stack Creation/Update:** the resources are created or updated in the specified order, forming a stack. The stack represents the complete set of provisioned resources.

Event-Driven Model: CloudFormation operates on an event-driven model. Events are generated during stack creation, update, or deletion processes, and CloudFormation logs these events. Users can monitor these events to track the progress of stack operations or identify any issues.

Rollback and Rollback Triggers: If an error occurs during the stack creation or update process, CloudFormation can automatically trigger a rollback, reverting the stack to its previous state. Rollback triggers can be defined in the template to specify conditions under which a rollback should occur.

Cross-Stack References: CloudFormation supports cross-stack references, allowing resources from one stack to refer to resources in another. This is useful for managing complex applications and dependencies that span multiple stacks.

Understanding the architecture of CloudFormation provides insights into how the service efficiently manages the orchestration and provisioning of AWS resources, ensuring a reliable and consistent infrastructure deployment process.

Answer the questions below

| What is generated during stack creation?

events

| Can you define rollback triggers in a template?

yay

| What allows you to refer to resources in another stack?

Cross-Stack References

Task 6 - CloudFormation Configuration & Use Cases

Configuration and Concepts: CloudFormation templates consist of key sections:

- **AWSTemplateFormatVersion:** Specifies the AWS CloudFormation template version.
- **Description:** Describes the template.
- **Parameters:** You can input custom values when creating or updating a stack.
- **Resources:** Defines the AWS resources to be created or managed.
- **Outputs:** Describes the values that can be queried once the stack is created or updated.

Intrinsic Functions: CloudFormation templates support "intrinsic functions", which help you perform operations. Examples of intrinsic functions are referencing resources, performing calculations, and conditionally including resources:

```
Resources:
  MyInstance:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: ami-12345678
      InstanceType: t2.micro

Outputs:
  InstanceId:
    Value: !Ref MyInstance

  PublicDnsName:
    Value: !GetAtt MyInstance.PublicDnsName

  SubstitutedString:
    Value: !Sub "Hello, ${MyInstance}"
```

- **Fn::Ref** : References the value of the specified resource.
- **Fn::GetAtt** : Gets the value of an attribute from a resource in the template.
- **Fn::Sub** : Performs string substitution.

Resource Dependencies: Resources in CloudFormation can have dependencies on each other. For example, an EC2 instance might depend on a VPC being created first. CloudFormation automatically handles the order of resource creation and updates based on these dependencies.

Change Sets: Before making changes to a stack, CloudFormation lets you preview the changes using a Change Sets feature. This helps in understanding the impact of modifications before they are applied.

Use Cases: There are many use cases for resource provisioning and management in AWS. Here are some examples:

Infrastructure Provisioning and Management: CloudFormation simplifies the process of creating and managing AWS resources. It ensures consistency and repeatability in infrastructure deployments, reducing the risk of configuration errors.

Application Lifecycle Management: CloudFormation is commonly used to manage the entire lifecycle of applications. This includes provisioning resources, deploying application code, and handling updates or rollbacks.

Multi-Environment Deployments: Deploying the same infrastructure across multiple environments (dev, test, production) is often necessary. CloudFormation allows you to use the same template with different parameter values for each environment.

Resource Scaling: As your application or workload grows, CloudFormation enables you to quickly scale your infrastructure by modifying the template or using auto-scaling capabilities.

CloudFormation provides a robust and scalable way to manage AWS resources through code, offering benefits in terms of automation, consistency, and collaboration in the cloud infrastructure setup and maintenance.

Answer the questions below

| Does CloudFormation support intrinsic functions?

yay

| What feature helps you understand the impact of modifications before they are applied?

change sets

Task 7 - Terraform vs CloudFormation

Terraform vs. CloudFormation



CloudFormation

- **Vendor Lock-In:** AWS CloudFormation is tightly integrated with AWS services, making it the natural choice for AWS-centric environments.
- **Native Integration:** Seamlessly integrates with other AWS services, providing direct resource references and a unified management experience.
- **Service Coverage:** Provides comprehensive coverage of AWS services, often including support for new services shortly after release.
- **Learning Curve:** Learning CloudFormation may be easier for those familiar with AWS services.

Terraform

- **Cloud-Agnostic:** Terraform is cloud-agnostic, supporting multiple cloud providers, including AWS, Azure, Google Cloud, and more. This makes it a versatile choice for hybrid or multi-cloud deployments.
- **Community and Modules:** This has a large and active community, and using modules allows for reusable and shareable code.

- **State Management:** Terraform uses a state file to track the current state of infrastructure, providing a clear view of the deployed resources.
- **Language Flexibility:** The HashiCorp Configuration Language (HCL) used by Terraform is designed for readability and is more flexible than JSON or YAML.

Use Cases

Terraform Use Cases

1. Multi-Cloud Environments: *Scenario:* A company utilises its infrastructure with multiple cloud providers such as AWS, Azure, and GCP. Reason for Terraform:

- Terraform has providers for various cloud platforms, making it easier to manage multi-cloud environments.
- A single Terraform configuration can span different cloud providers, allowing consistent infrastructure as code (IaC) across the entire infrastructure landscape.

2. Community Modules and Providers: *Scenario:* An organisation heavily relies on community-contributed modules and providers to enhance infrastructure provisioning. Reason for Terraform:

- Terraform has a vibrant community that contributes reusable modules and providers for a wide range of services and platforms.
- Leveraging community-driven content can significantly speed up the development process and provide best practices adopted by a broader audience.

AWS CloudFormation Use Cases

1. Deep AWS Integration: *Scenario:* An enterprise's infrastructure is predominantly AWS-centric, and the team is deeply invested in AWS-specific services and features. Reason for CloudFormation:

- AWS CloudFormation provides native integration with AWS services, making it well-suited for managing AWS-specific resources.
- Features like CloudFormation StackSets and Change Sets enhance the management of AWS-specific resources and deployments.

2. Managed Service Integration: *Scenario:* An organisation heavily utilises AWS-managed services, such as AWS Elastic Beanstalk or AWS OpsWorks. Reason for CloudFormation:

- AWS CloudFormation integrates seamlessly with AWS-managed services, offering simplified provisioning and management of these services.
- CloudFormation provides specific resource types for many AWS-managed services, allowing for a higher level of abstraction in the templates.

Answer the questions below

| Is CloudFormation cloud agnostic?

no

| Which IaC tool has deep AWS integration?

CloudFormation

| Which IaC tool has accessible community-driven modules?

Terraform

Task 8 - Secure IaC

Secure Cloud IaC Best Practices

For Both CloudFormation and Terraform

- **Version Control:** Store IaC code in version control systems like Git to track changes, facilitate collaboration, and maintain a version history.
- **Least Privilege Principle:** always assign the least permissions and scope for credentials and IaC tools. Only grant the needed permissions for the actions to be performed.
- **Parameterise Sensitive Data:** Use parameterisation to handle credentials or API keys and avoid hardcoding secrets directly into the IaC code.
- **Secure Credential Management:** leverage the cloud platform's secure credential management solutions or services to securely handle and store sensitive information, e.g., vaults for secret management.

- **Audit Trails:** enable logging and monitoring features to maintain an audit trail of changes made through IaC tools. Use these logs to conduct reviews periodically.
- **Code Reviews:** implement code reviews to ensure IaC code adheres to best security practices. Collaborative review processes can catch potential security issues early.

For AWS CloudFormation

- **Use IAM Roles:** Assign Identity and Access Management (IAM) roles with the minimum required permissions to CloudFormation stacks. Avoid using long-term access keys when possible.
- **Secure Template Storage:** store CloudFormation templates in an encrypted S3 bucket and restrict access to only authorised users or roles.
- **Stack Policies:** implement stack policies to control updates to stack resources and enforce specific conditions during updates.

For Terraform

- **Backend State Encryption:** enable backend state encryption to protect sensitive information stored in the Terraform state file.
- **Use Remote Backends:** store the Terraform state remotely using backends like Amazon S3 or Azure Storage. This enhances collaboration and provides better security.
- **Variable Encryption:** consider encrypting sensitive values using tools like HashiCorp Vault or other secure key management solutions when using variables.
- **Provider Configuration:** Securely configure provider credentials using environment variables, variable files, or other secure methods.

Answer the questions below

| What can you do instead of hardcoding secrets in IaC code?

Parameterise Sensitive Data

| What collaborative process can catch security issues early?

Code Reviews

| What policies can you implement in CloudFormation for update controls?

Stack Policies

Task 9 - Practical

Okay, so we've gone through cloud-based IaC, covering Terraform and AWS CloudFormation. Now it's time to give you a practical demonstration of what your day-to-day might look like if you were a DevSecOps engineer using these tools. In this static site, you will be presented with a block of text; this text represents your infrastructure. Your infrastructure as code! Initially, you will be shown a rather insecure infrastructure, barebones with little security in place. You will then be given the chance to choose between resource blocks to add or variables to change, which will secure this infrastructure. Once the infrastructure has been secured and your role as a DevSecOps engineer is fulfilled, the flag is all yours! To begin your cloud-based adventure, click the green 'View Site' button in the top right of this task.

1. Introduction Menu: Upon launching the static site, you'll encounter an introduction menu. This menu provides options such as resetting infrastructure, accessing instructions, and building infrastructure. Choose your desired option to proceed.
2. IaC Screen : After selecting an option from the introduction menu, you'll transition to an IaC config screen. This screen simulates the stages of Terraform: init, plan, and apply.
3. Instruction Model: The static site acts as an instructional model, guiding you through the fundamentals of YAML configuration. It adopts a user-friendly approach inspired by Scratch and Code Warrior methodologies. Learn without worrying about intricate code details.
4. Interactive Parts: During the exercise, you'll interact with clickable blanks in a YAML config file. These blanks represent components to be added to enhance infrastructure security. Choose the correct options to fill in the blanks.
5. Criteria for Flag: The exercise comprises three phases: Review, Implement, and Destroy. Successfully completing each phase will light up a diamond at the bottom

of the screen.

6. Messages:

- **Welcome Message:** At the start, you'll receive a welcoming message.
- **Review Phase Message:** Analyse a YAML config file for security gaps and click "Next" when ready.
- **Implement Phase Message:** Fill in blanks to enhance infrastructure. Initiate Terraform workflow and follow instructions.
- **Destroy Phase Message:** After successful implementation, destroy the infrastructure. Click the "Terraform Destroy" button.

7. Config Files: Utilise the provided YAML config file for infrastructure representation. During the Implement phase, interact with clickable blanks using dummy options.

Answer the questions below

| Can you use your Cloud-based IaC knowledge to get the flag?

THM{c10uD-b@z3d-1@SeE}